

```
In [2]: %matplotlib inline
import jax.numpy as jnp
from jax import random
import matplotlib.pyplot as plt
import matplotlib.colors as colors
import seaborn as snb

from exercise6 import compute_err
from exercise6 import eval_density_grid
from exercise6 import load_MNIST_subset
from exercise6 import plot_with_uncertainty
from exercise6 import add_colorbar
from exercise6 import StationaryIsotropicKernel
from exercise6 import squared_exponential
from exercise6 import matern12
from exercise6 import matern32
from exercise6 import NeuralNetworkMAP
from exercise6 import generate_samples

# from autograd import grad,
from scipy.optimize import minimize
from scipy.stats import norm
from scipy.stats import binom as binom_dist

from jax import config
config.update("jax_enable_x64", True)

snb.set_style('darkgrid')
snb.set_theme(font_scale=1.1)
```

02477 Bayesian Machine Learning: Gaussian processes classification

The purpose of this exercise is to

- get familiar with Bayesian modelling using non-Gaussian likelihoods, we will study the Bernoulli likelihood in particular.
- study Gaussian processes for binary classification via the Laplace approximation
- dive deeper into Gaussian processes

Content

- Part 1: Theory for binary classification using Gaussian process
- Part 2: Implementing the Laplace approximation (Step 1 of 3)
- Part 3: Implementing the posterior distribution of $p(f^* | y, x^*)$ (step 2 of 3)
- Part 4: Compute the posterior predictive distribution of the label y^* for the input point x^* , i.e. to compute $p(y^* | y, x^*)$. (Step 3 of 3)
- Part 5: Gaussian process classification for 2D data set

Note: In this exercise, we will use the code from last week for the squared exponential kernel etc. We will also make use of a set of auxiliary functions, which are less important for the learning objectives of today and therefore, they can found in the module `exercise6.py`.

A toy dataset

First, we will implement and test everything on a simple 1D toy dataset. The dataset is generated using the following model:

$$y_n | x_n \sim \text{Ber}(\sigma(f(x_n)))$$

$$f(x_n) = 5 \sin\left(\frac{3}{4}x\right)$$

$$x_n \sim \text{N}(0, 2.5),$$

Hence, the true data generating process is $p(y = 1 | x) = \sigma(5 \sin(\frac{3}{4}x))$, where $\sigma(x) = \frac{1}{1 + e^{-x}}$ is the logistic sigmoid function.

```
In [12]: # fix seed
seed = 2
key = random.PRNGKey(seed)
key_X, key_y = random.split(key)

sigmoid = lambda x: 1./(1+jnp.exp(-x))

# true p(y=1|x) = sigmoid(f(x))
```

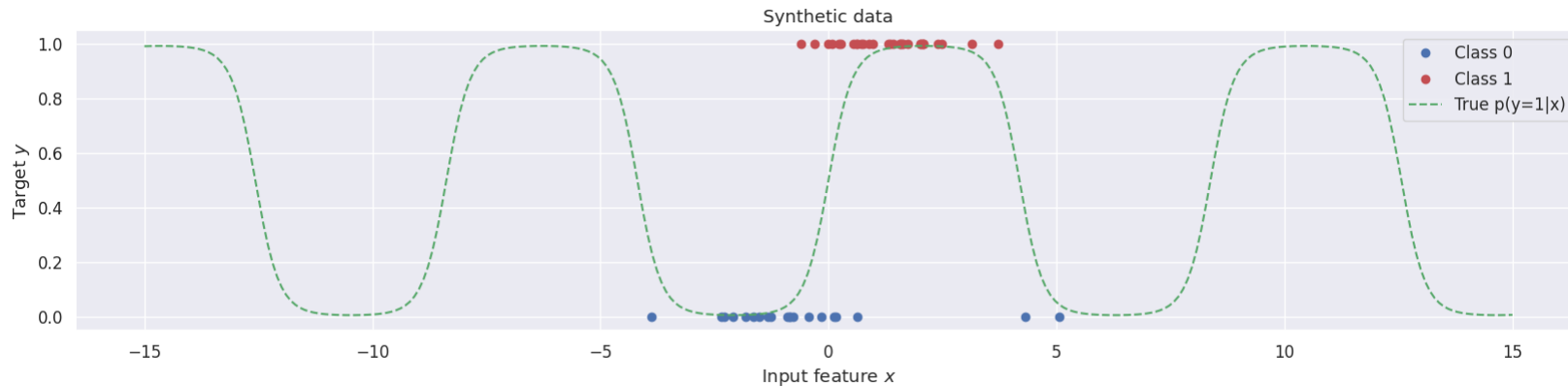
```
f = lambda x: 5*jnp.sin(0.75*x)

# sample data
N = 50
X = jnp.sort(jnp.sqrt(2.5)*random.normal(key_X, shape=(N, ))[:, None])
y = random.binomial(key_y, 1, sigmoid(f(X)))

# define points for prediction/plotting
Xstar = jnp.linspace(-15, 15, 300)[:, None]

# plot data
def plot_data(ax, X, y, title="Synthetic data", plot_true=False):
    ax.plot(X[y==0], y[y==0], 'bo', label='Class 0')
    ax.plot(X[y==1], y[y==1], 'ro', label='Class 1')
    if plot_true:
        ax.plot(Xstar, sigmoid(f(Xstar)), 'g--', label='True p(y=1|x)')
    ax.set(xlabel='Input feature $x$', ylabel='Target $y$', title=title)
    ax.legend()

fig, ax = plt.subplots(1, 1, figsize=(20, 4))
plot_data(ax, X, y, plot_true=True)
```



Part 1: Theory for binary classification using Gaussian process

We will study Gaussian processes for binary classification using the following model

$$y | f(\mathbf{x}) \sim \text{Ber}[\sigma(f(\mathbf{x}))] = \prod_{n=1}^N \sigma(f_n)^{y_n} (1 - \sigma(f_n))^{1-y_n}, \text{ where } \sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$f(\mathbf{x}) \sim \text{GP}(0, k(\mathbf{x}, \mathbf{x}')),$$

where $k(\mathbf{x}, \mathbf{x}')$ is the covariance function and $\sigma: \mathbb{R} \rightarrow (0, 1)$ is a suitable inverse link function. Here we will use the logistic sigmoid function. The purpose of the function $\sigma: \mathbb{R} \rightarrow (0, 1)$ is to "squeeze" the values of $y(\mathbf{x})$ from the entire real line to the unit interval such that we can interpret $\sigma(f(\mathbf{x})) \in (0, 1)$ as a probability for all $\mathbf{x}$.

Our goal is to compute the *posterior predictive distribution* $p(y^* = 1 | y, \mathbf{x}^*)$ for the class label y^* associated with a new input point $\mathbf{x}^* \in \mathbb{R}^D$ given a dataset $D = \{\mathbf{x}_n, y_n\}_{n=1}^N$ for $\mathbf{x} \in \mathbb{R}^D$ and $y_n \in \{0, 1\}$. Let $\mathbf{y} \in \{0, 1\}^N$ be the vector of binary observations and let $\mathbf{f} = [f(\mathbf{x}_1) \ f(\mathbf{x}_2) \ \dots \ f(\mathbf{x}_N)] \in \mathbb{R}^N$ be the corresponding vector of latent function values for each sample in the training set, then the joint model of the observations $\mathbf{y}$ and the latent function values $\mathbf{f}$ becomes

$$p(\mathbf{y}, \mathbf{f}) = p(\mathbf{y} | \mathbf{f}) p(\mathbf{f}) = \prod_{n=1}^N \text{Ber}(y_n | \sigma(f_n)) \mathcal{N}(\mathbf{f} | \mathbf{0}, \mathbf{K}),$$

where we have assumed the observations to be conditionally independent given $\mathbf{f}$, and $\mathbf{K} \in \mathbb{R}^{N \times N}$ is the prior covariance matrix of $\mathbf{f}$, i.e. $\mathbf{K}_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$.

A 3-step approach for computing the posterior predictive distribution for y^*

Just like for the Bayesian logistic regression model, exact analytical Bayesian inference is intractable for this model, and therefore, we will resort to the Laplace approximation. Making predictions for a Gaussian process classification model using the Laplace approximation typically requires three steps:

- Step 1: Approximating the posterior distribution $p(\mathbf{f} | \mathbf{y})$
- Step 2: Computing the posterior distribution of the latent function evaluated at a new point $\mathbf{x}^*$, i.e. computing $p(f^* | y, \mathbf{x}^*)$ where $f^* = f(\mathbf{x}^*)$.

- Step 3: Computing the posterior predictive distribution of the label y^* for the input point $\mathbf{x}^*$, i.e. computing $p(y^* | \mathbf{y}, \mathbf{x}^*)$.

Note: For the regression models with Gaussian likelihoods, there is often a lot of confusion about the difference between f^* and y^* . Part of this confusion is often caused by the fact that for regression f^* and y^* typically belong to the same mathematical space, e.g. $f^*, y^* \in \mathbf{R}$ and often f^* can be interpreted as a *denoised* version of y^* . However, for binary classification models, the two quantities belong to different spaces, i.e. $f^* \in \mathbf{R}$ and $y^* \in \{0, 1\}$ and here it might be more clear that $f(\mathbf{x})$ simply is a latent function, which we introduce in order to model the distribution $\text{Ber}(y^* | \sigma(f^*))$.

Gaussian Processes for Binary Classification (Logistic Link)

We study GP classification with a logistic link:

$$y_n \mid f_n \sim \text{Ber}[\sigma(f_n)], \quad \sigma(a) = \frac{1}{1 + \exp(-a)}.$$

For a dataset $\mathbf{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$ with $\mathbf{x}_n \in \mathbf{R}^D, y_n \in \{0, 1\}$, the likelihood factorizes as:

$$p(\mathbf{y} \mid \mathbf{f}) = \prod_{n=1}^N \sigma(f_n)^{y_n} [1 - \sigma(f_n)]^{1-y_n},$$

where $\mathbf{f} = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)]^\top$.

The GP prior is:

$$f(\cdot) \sim \text{GP}(0, k(\cdot, \cdot)) \quad \Rightarrow \quad \mathbf{f} \sim \mathcal{N}(\mathbf{0}, \mathbf{K}), \quad \mathbf{K}_{nm} = k(\mathbf{x}_n, \mathbf{x}_m).$$

The role of $\sigma: \mathbf{R} \rightarrow (0, 1)$ is to squash latent real values to probabilities so that $\sigma(f(\mathbf{x}))$ can be interpreted as $\mathbb{P}(y = 1 \mid \mathbf{x})$.

Our goal is the posterior predictive for a new input $\mathbf{x}_*$:

$$p(y_* = 1 \mid \mathbf{y}, \mathbf{x}_*).$$

Note: $f_* = f(\mathbf{x}_*) \in \mathbf{R}$ (latent) vs. $y_* \in \{0, 1\}$ (observed label).

A 3-Step Approach (Laplace Approximation)

Exact inference is intractable with the logistic likelihood, so we use the **Laplace approximation**.

1. Approximate $p(\mathbf{f} \mid \mathbf{y})$ with a Gaussian by expanding the log posterior around its mode.
2. Obtain $p(f_* \mid \mathbf{y}, \mathbf{x}_*)$ from the GP prior conditioned on the approximate posterior over $\mathbf{f}$.
3. Predict $p(y_* \mid \mathbf{y}, \mathbf{x}_*)$ by integrating the sigmoid against the Gaussian in Step 2.

Why an Approximation is Needed (One Line of Math)

Define:

$$\Psi(\mathbf{f}) \equiv \log p(\mathbf{y} \mid \mathbf{f}) - \frac{1}{2} \mathbf{f}^\top \mathbf{K}^{-1} \mathbf{f}, \quad \log p(\mathbf{y} \mid \mathbf{f}) = \sum_{n=1}^N \left[y_n \log \sigma(f_n) + (1 - y_n) \log(1 - \sigma(f_n)) \right].$$

Let $\pi = \sigma(\mathbf{f})$ (elementwise) and $\mathbf{W} = \text{diag}(\pi_n(1 - \pi_n))$. Then:

$$\nabla \Psi(\mathbf{f}) = \mathbf{y} - \pi - \mathbf{K}^{-1} \mathbf{f}, \quad \nabla^2 \Psi(\mathbf{f}) = -(\mathbf{K}^{-1} + \mathbf{W}).$$

Because $\log p(\mathbf{y} \mid \mathbf{f})$ is not quadratic, the posterior is not Gaussian.

Intuition: $\mathbf{W}_{nn} = \pi_n(1 - \pi_n)$ peaks at 0.5 and is tiny near 0 or 1; very certain points contribute little curvature.

Step 1: Laplace Approximation to $p(\mathbf{f} \mid \mathbf{y})$

- **Mode (mean of approximation):**

$$\hat{\mathbf{f}} = \arg \max_{\mathbf{f}} \Psi(\mathbf{f}).$$

- **Numerically stable Newton step** (avoids explicit $\mathbf{K}^{-1}$):

$$(\mathbf{K}^{-1} + \mathbf{W}) \mathbf{f}_{\text{new}} = \mathbf{W} \mathbf{f} + (\mathbf{y} - \pi).$$

- In practice: define $\mathbf{B} = \mathbf{I} + \mathbf{W}^{1/2}\mathbf{K}\mathbf{W}^{1/2}$ and use Cholesky.
- **Covariance (negative Hessian inverse at the mode):**

$$\Sigma = (\mathbf{K}^{-1} + \mathbf{W})^{-1} = \mathbf{K} - \mathbf{K}\mathbf{W}^{1/2}\mathbf{B}^{-1}\mathbf{W}^{1/2}\mathbf{K}.$$

Step 2: Posterior Over the Latent f_*

From the GP prior:

$$\begin{bmatrix} \mathbf{f} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \mathbf{k}_* \\ \mathbf{k}_*^\top & k_{**} \end{bmatrix}\right),$$

where:

- $\mathbf{k}_* = [k(\mathbf{x}_1, \mathbf{x}_*), \dots, k(\mathbf{x}_N, \mathbf{x}_*)]^\top$
- $k_{**} = k(\mathbf{x}_*, \mathbf{x}_*)$

Using the approximation $\mathbf{f} \sim \mathcal{N}(\hat{\mathbf{f}}, \Sigma)$, we get:

$$f_* \mid \mathbf{y}, \mathbf{x}_* \approx \mathcal{N}(m_*, v_*),$$

with:

$$\begin{aligned} m_* &= \mathbf{k}_*^\top \mathbf{K}^{-1} \hat{\mathbf{f}}, \\ v_* &= k_{**} - \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{k}_* + \mathbf{k}_*^\top \mathbf{K}^{-1} \Sigma \mathbf{K}^{-1} \mathbf{k}_*. \end{aligned}$$

Numerically stable version using $\mathbf{B}$:

$$v_* = k_{**} - (\mathbf{W}^{1/2} \mathbf{k}_*)^\top \mathbf{B}^{-1} (\mathbf{W}^{1/2} \mathbf{k}_*).$$

Step 3: Posterior Predictive for y_*

We must integrate the sigmoid against the Gaussian:

- For the logistic link, no closed form.
- Highly accurate approximation:

$$p(y_* = 1 \mid \mathbf{y}, \mathbf{x}_*) = \int \sigma(f_*) \mathcal{N}(f_*; m_*, v_*) df_*.$$

$$p(y_* = 1 \mid \mathbf{y}, \mathbf{x}_*) \approx \sigma\left(\frac{m_*}{\sqrt{1 + \frac{\pi}{8} v_*}}\right).$$

Alternatively: Use 1D Gauss–Hermite quadrature for numerical integration.

Probit Aside (Intuition)

If we used a probit link $\Phi(\cdot)$ instead of $\sigma(\cdot)$, then:

$$p(y_* = 1 \mid \mathbf{y}, \mathbf{x}_*) = \Phi\left(\frac{m_*}{\sqrt{1 + v_*}}\right).$$

Interpreting the Model

- **Decision surface for 0.5 threshold:** zero-level set $f(\mathbf{x}) = 0$, since σ is monotone.
- **Effect of kernel hyperparameters:**

- Long length-scales $\Rightarrow$ smoother decision surfaces.
- Larger output scale $\Rightarrow$ more confident (smaller $v_{\star}$).
- **Linear kernel:** GP classification reduces to Bayesian logistic regression.

(Optional) Hyperparameter Learning via Laplace Evidence

Maximize Laplace approximation to the log marginal likelihood:

$$\log p(\mathbf{y} \mid \boldsymbol{\theta}) \approx -\frac{1}{2} \hat{\mathbf{f}}^{\top} \mathbf{K}^{-1} \hat{\mathbf{f}} + \log p(\mathbf{y} \mid \hat{\mathbf{f}}) - \frac{1}{2} \log |\mathbf{B}| + \text{const}, \quad \mathbf{B} = \mathbf{I} + \mathbf{W}^{1/2} \mathbf{K} \mathbf{W}^{1/2}.$$

Sanity Checks & Numerics

- **Shapes:** $\mathbf{K} \in \mathbb{R}^{N \times N}$, $\mathbf{k}_{\star} \in \mathbb{R}^N$, $k_{\star, \star} \in \mathbb{R}$.
- **Avoid** forming $\mathbf{K}^{-1}$ explicitly — use Cholesky decompositions.
- **Add jitter** $\epsilon \mathbf{I}$ to $\mathbf{K}$ if needed.
- When $|f_n|$ is large, $\mathbf{W}_{nn} \approx 0$: very certain points don't dominate curvature $\Rightarrow$ better stability.

Summary (Implementation Formulas)

MAP: $(\mathbf{K}^{-1} + \mathbf{W})\mathbf{f}_{\text{new}} = \mathbf{W}\mathbf{f} + \mathbf{y} - \boldsymbol{\pi}$

Covariance: $\boldsymbol{\Sigma} = \mathbf{K} - \mathbf{K}\mathbf{W}^{1/2} \mathbf{B}^{-1} \mathbf{W}^{1/2} \mathbf{K}$, $\mathbf{B} = \mathbf{I} + \mathbf{W}^{1/2} \mathbf{K} \mathbf{W}^{1/2}$

Posterior mean: $\mathbf{m}_{\star} = \mathbf{k}_{\star}^{\top} \mathbf{K}^{-1} \hat{\mathbf{f}}$

Posterior variance: $v_{\star, \star} = k_{\star, \star} - (\mathbf{W}^{1/2} \mathbf{k}_{\star})^{\top} \mathbf{B}^{-1} (\mathbf{W}^{1/2} \mathbf{k}_{\star})$

$$\text{Prediction: } p(y_{\star} = 1 \mid \cdot) \approx \sigma \left(\frac{\mathbf{m}_{\star}}{\sqrt{1 + \frac{\pi}{8} v_{\star, \star}}} \right)$$

Part 2: Implementing the Laplace approximation (Step 1 of 3)

Our goal is now to implement the Laplace approximation for $p(\mathbf{f} \mid \mathbf{y})$ such that

$$p(\mathbf{f} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{f}) p(\mathbf{f})}{p(\mathbf{y})} \approx q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, \mathbf{S}),$$

where $\mathbf{m} = \mathbf{f}_{\text{MAP}}$ is the MAP estimator for $\mathbf{f}$ and $\mathbf{S}$ is the negative inverse Hessian. As usual, we set-up the log joint distribution

$$\log p(\mathbf{y}, \mathbf{f}) = \log p(\mathbf{y} \mid \mathbf{f}) + \log p(\mathbf{f}) = \sum_{n=1}^N \log p(y_n \mid f_n) - \frac{N}{2} \log(2\pi) - \frac{1}{2} |\mathbf{K}| - \frac{1}{2} \mathbf{f}^{\top} \mathbf{K}^{-1} \mathbf{f}$$

where the gradient and the Hessian are given by

$$\nabla_{\mathbf{f}} \log p(\mathbf{y}, \mathbf{f}) = \sum_{n=1}^N \nabla_{\mathbf{f}} \log p(y_n \mid f_n) - \frac{1}{2} \nabla_{\mathbf{f}} \mathbf{f}^{\top} \mathbf{K}^{-1} \mathbf{f} = \mathbf{g} - \mathbf{K}^{-1} \mathbf{f},$$

$$\nabla_{\mathbf{f}}^2 \log p(\mathbf{y}, \mathbf{f}) = \sum_{n=1}^N \nabla_{\mathbf{f}}^2 \log p(y_n \mid f_n) - \frac{1}{2} \nabla_{\mathbf{f}}^2 \mathbf{f}^{\top} \mathbf{K}^{-1} \mathbf{f} = \sum_{n=1}^N \nabla_{\mathbf{f}}^2 \log p(y_n \mid f_n) - \mathbf{K}^{-1} = -\boldsymbol{\Lambda} - \mathbf{K}^{-1},$$

where $\mathbf{g} \in \mathbb{R}^N$ with entries given by $g_n = \frac{\partial}{\partial f_n} \log p(y_n \mid f_n)$ and $\boldsymbol{\Lambda} \in \mathbb{R}^{N \times N}$ is a diagonal matrix with elements $\Lambda_{nn} = -\nabla_{f_n}^2 \log p(y_n \mid f_n)$ evaluated at the mode $\hat{\mathbf{f}}_{\text{MAP}} = \text{argmax}_{\mathbf{f}} \log p(\mathbf{y}, \mathbf{f})$. Thus, the mean and covariance matrix for the Laplace approximation $q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, \mathbf{S})$ become

$$\mathbf{m} = \hat{\mathbf{f}}_{\text{MAP}} \\ \mathbf{S} = \left(\mathbf{K}^{-1} + \boldsymbol{\Lambda} \right)^{-1}$$

The expression for the posterior covariance matrix $\mathbf{S}$ involves the inverse of $\mathbf{K}$, which we know can be numerically unstable. To avoid having to compute the inverse of $\mathbf{K}$ explicitly, we will use the [Woodbury matrix identity](#) to re-write the expression for the posterior covariance as

$$\mathbf{S} = \left(\mathbf{K}^{-1} + \mathbf{\Lambda} \right)^{-1} = \mathbf{K} - \mathbf{K} \mathbf{\Lambda} \frac{1}{2} \left(\mathbf{I} + \mathbf{\Lambda} \frac{1}{2} \mathbf{K} \mathbf{\Lambda} \frac{1}{2} \right)^{-1} \mathbf{\Lambda} \frac{1}{2} \mathbf{K}$$

$\mathbf{\tilde{B}}$

This might seem like a complicated way of calculating something simple, but the key thing is that the matrix $\mathbf{B} = \mathbf{I} + \mathbf{\Lambda} \frac{1}{2} \mathbf{K} \mathbf{\Lambda} \frac{1}{2}$ is usually well-behaved because its eigenvalues are lowerbounded by 1, and hence, the linear system $\mathbf{B}^{-1} \mathbf{\Lambda} \frac{1}{2} \mathbf{K}$ can usually be solved without problems, e.g. via Cholesky decompositions.

The contributions of the likelihood to the gradient and Hessian

By inspecting the equations above, we see that both the gradient and Hessian contains a contribution from the prior and a contribution from the likelihood. First, will focus on implementing the contributions from the likelihood. We need to compute the first and second order derivatives of the log likelihood $\log p(y_n | f_n) = \log \text{Ber}(y_n | \sigma(f_n))$

Part 2: Implementing the Laplace approximation (Step 1 of 3)

Our goal is to approximate the intractable posterior $p(\mathbf{f} \mid \mathbf{y})$ with a Gaussian:

$$p(\mathbf{f} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{f}) p(\mathbf{f})}{p(\mathbf{y})} \approx q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, \mathbf{S}),$$

where $\mathbf{m} = \hat{\mathbf{f}}_{\text{MAP}}$ is the maximizer (mode) of the log posterior and $\mathbf{S}$ is the negative inverse Hessian at the mode.

Log joint, gradient, and Hessian

The log joint is

$$\log p(\mathbf{y}, \mathbf{f}) = \sum_{n=1}^N \log p(y_n \mid f_n) - \frac{1}{2} \mathbf{f}^\top \mathbf{K}^{-1} \mathbf{f} - \frac{1}{2} \log |\mathbf{K}| - \frac{N}{2} \log(2\pi).$$

Define the likelihood terms for the logistic link

$$\log p(\mathbf{y} \mid \mathbf{f}) = \sum_{n=1}^N [y_n \log \sigma(f_n) + (1 - y_n) \log(1 - \sigma(f_n))], \quad \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Let $\pi = \sigma(\mathbf{f})$ elementwise, and define

$$\mathbf{g} = \nabla_{\mathbf{f}} \log p(\mathbf{y} \mid \mathbf{f}) = \mathbf{y} - \boldsymbol{\pi} \in \mathbb{R}^N, \quad \mathbf{\Lambda} = -\nabla_{\mathbf{f}}^2 \log p(\mathbf{y} \mid \mathbf{f}) = \text{diag}(\pi_n(1 - \pi_n)) \in \mathbb{R}^{N \times N}.$$

Then

$$\nabla_{\mathbf{f}} \log p(\mathbf{y}, \mathbf{f}) = \mathbf{g} - \mathbf{K}^{-1} \mathbf{f}, \quad \nabla_{\mathbf{f}}^2 \log p(\mathbf{y}, \mathbf{f}) = -(\mathbf{K}^{-1} + \mathbf{\Lambda}).$$

Intuition: $\Lambda_{nn} = \pi_n(1 - \pi_n)$ is largest near 0.5 and tiny near 0 or 1, so highly certain points contribute little curvature.

MAP estimate and Newton update

The mode $\hat{\mathbf{f}}_{\text{MAP}}$ solves $\nabla_{\mathbf{f}} \log p(\mathbf{y}, \mathbf{f}) = \mathbf{0}$.

A stable Newton step is obtained by solving the linear system

$$(\mathbf{K}^{-1} + \mathbf{\Lambda}) \mathbf{f}_{\text{new}} = \mathbf{\Lambda} \mathbf{f} + \mathbf{g} \quad (\text{with } \mathbf{g} = \mathbf{y} - \boldsymbol{\pi}, \mathbf{\Lambda} = \text{diag}(\pi_n(1 - \pi_n))).$$

Iterate until convergence (e.g., change in $\mathbf{f}$ or in the log posterior below a tolerance).

A numerically robust way to apply the left-hand side avoids forming $\mathbf{K}^{-1}$.

Multiply by $\mathbf{K}$ and use the identity below to solve via Cholesky of

$$\mathbf{B} = \mathbf{I} + \mathbf{\Lambda}^{1/2} \mathbf{K} \mathbf{\Lambda}^{1/2}.$$

One convenient explicit form for the Newton update is

$$\mathbf{f}_{\text{new}} = [\mathbf{K} - \mathbf{K} \mathbf{\Lambda}^{1/2} \mathbf{B}^{-1} \mathbf{\Lambda}^{1/2} \mathbf{K}] (\mathbf{g} + \mathbf{\Lambda} \mathbf{f}).$$

Laplace posterior $q(\mathbf{f})$

At the converged mode $\hat{\mathbf{f}}_{\text{MAP}}$ (with $\hat{\boldsymbol{\pi}} = \sigma(\hat{\mathbf{f}}_{\text{MAP}})$ and $\hat{\boldsymbol{\Lambda}} = \text{diag}(\hat{\pi}_n(1 - \hat{\pi}_n))$),

$$\mathbf{m} = \hat{\mathbf{f}}_{\text{MAP}}, \quad \mathbf{S} = (\mathbf{K}^{-1} + \hat{\boldsymbol{\Lambda}})^{-1}.$$

To avoid explicitly inverting $\mathbf{K}$, use the Woodbury identity to compute $\mathbf{S}$ as

$$\mathbf{S} = \mathbf{K} - \mathbf{K} \hat{\boldsymbol{\Lambda}}^{1/2} (\mathbf{I} + \hat{\boldsymbol{\Lambda}}^{1/2} \mathbf{K} \hat{\boldsymbol{\Lambda}}^{1/2})^{-1} \hat{\boldsymbol{\Lambda}}^{1/2} \mathbf{K}.$$

$\mathbf{B}$

The matrix $\mathbf{B}$ is typically well-conditioned: since $\hat{\boldsymbol{\Lambda}} \succeq \mathbf{0}$, all eigenvalues of $\mathbf{B} = \mathbf{I} + \hat{\boldsymbol{\Lambda}}^{1/2} \mathbf{K} \hat{\boldsymbol{\Lambda}}^{1/2}$ are ≥ 1 , so linear solves with $\mathbf{B}$ via Cholesky are stable.

Implementation notes (shapes & numerics)

- Shapes: $\mathbf{K} \in \mathbb{R}^{N \times N}$, $\boldsymbol{\Lambda}, \mathbf{B} \in \mathbb{R}^{N \times N}$, $\mathbf{f}, \mathbf{g} \in \mathbb{R}^N$.
- Never form $\mathbf{K}^{-1}$ explicitly. Use Cholesky solves for $\mathbf{K}$ and $\mathbf{B}$; add small jitter $\epsilon \mathbf{I}$ to $\mathbf{K}$ if needed.
- Update $\boldsymbol{\pi} = \sigma(\mathbf{f})$, $\boldsymbol{\Lambda} = \text{diag}(\pi_n(1 - \pi_n))$, $\mathbf{g} = \mathbf{y} - \boldsymbol{\pi}$ each Newton step.

(Reference box: formulas to implement)

Gradient: $\mathbf{g} = \mathbf{y} - \boldsymbol{\pi}$, $\boldsymbol{\pi} = \sigma(\mathbf{f})$,
Hessian (neg.): $\boldsymbol{\Lambda} = \text{diag}(\pi_n(1 - \pi_n))$,
Newton step: $(\mathbf{K}^{-1} + \boldsymbol{\Lambda}) \mathbf{f}_{\text{new}} = \boldsymbol{\Lambda} \mathbf{f} + \mathbf{g} \iff \mathbf{f}_{\text{new}} = [\mathbf{K} - \mathbf{K} \boldsymbol{\Lambda}^{1/2} \mathbf{B}^{-1} \boldsymbol{\Lambda}^{1/2} \mathbf{K}](\mathbf{g} + \boldsymbol{\Lambda} \mathbf{f})$,
 $\mathbf{B} = \mathbf{I} + \boldsymbol{\Lambda}^{1/2} \mathbf{K} \boldsymbol{\Lambda}^{1/2}$,
Laplace mean/cov: $\mathbf{m} = \hat{\mathbf{f}}_{\text{MAP}}$, $\mathbf{S} = (\mathbf{K}^{-1} + \hat{\boldsymbol{\Lambda}})^{-1} = \mathbf{K} - \mathbf{K} \hat{\boldsymbol{\Lambda}}^{1/2} \mathbf{B}^{-1} \hat{\boldsymbol{\Lambda}}^{1/2} \mathbf{K}$.

Task 2.1: Show that first and second order derivative of $\log p(y_n | f_n)$ is given by $\frac{\partial}{\partial f_n} \log p(y_n | f_n) = y_n - \sigma(f_n)$ and $\frac{\partial^2}{\partial f_n^2} \log p(y_n | f_n) = -\sigma(f_n)(1 - \sigma(f_n))$.

Hints: Recall the identities $\frac{\partial}{\partial f} \sigma(f) = \sigma(f)(1 - \sigma(f))$ and $\frac{\partial}{\partial x} \log(x) = \frac{1}{x}$.

Next, we will implement the contribution from the likelihood in the class `BernoulliLikelihood` below.

Task 2.2: Complete the implementation of the functions `log_lik`, `grad`, and `hessian` of the class below.

Hints: Each function should be fairly straight forward and consist of 1-2 lines of code each. Regarding the likelihood, you can either implement it from scratch or use the relevant module from `scipy.stats`

$$y | f(\mathbf{x}) \sim \text{Ber}[\sigma(f(\mathbf{x}))] = \prod_{n=1}^N \sigma(f_n)^{y_n} (1 - \sigma(f_n))^{1-y_n}, \text{ where } \sigma(x) = \frac{1}{1 + \exp(-x)}$$
$$f(\mathbf{x}) \sim \text{GP}(0, k(\mathbf{x}, \mathbf{x}')),$$

```
In [13]: from jax.scipy.linalg import cho_solve, cho_factor
from jax.scipy.special import logsumexp, expit, logit

class BernoulliLikelihood(object):
    """ Implement the Bernoulli likelihood with the sigmoid as inverse link function """

    def __init__(self, y):
        # store data & force shape (N, )
        self.y = y.ravel()

    def log_lik(self, f):
        """ Implements log p(y|f) = sum log p(y_n|f_n), where p(y_n|f_n) = Ber(y_n|sigmoid(f_n)).

        Argument:
        f          --      vector of function values, shape (N, )

        Returns
        ll         --      sum of log likelihoods for all N data points, scalar
        """
        # Your solution goes here
        p = sigmoid(f)
        ll = jnp.sum(
            self.y * jnp.log( p ) + ( 1 - self.y ) * jnp.log( 1 - p )
        )
```

```

        )
#####
# End of solution
#####

# check shape and return
assert ll.shape == (), f"Expected shape for loglik_is (), but the actual shape was {ll.shape}. Please check implementation"
return ll

def grad(self, f):
    """ Implements the gradient of log p(y|n)

    Argument:
    f      --      vector of function values, shape (N, )

    Returns
    g      --      gradient of log p(y|f), i.e. a vector of first order derivatives with shape (N, )

    """
#####
# Your solution goes here
#####

g = self.y - sigmoid(f)

#####
# End of solution
#####
# check shape and return
assert g.shape == (len(f), ), f"Expected shape for g is {(len(f)}, ), but the actual shape was {g.shape}. Please check implementation"
return g

def hessian(self, f):
    """ Implements the Hessian of log p(y|n)

    Argument:
    f      --      vector of function values, shape (N, )

    Returns:
    Lambda --      Hessian of likelihood, i.e. a diagonal matrix with the second order derivatives on the diagonal, shape (N, N)
    """
#####
# Your solution goes here
#####
p = sigmoid(f)
Lambda = jnp.diag(- p * (1 - p))
#####
# End of solution
#####

# check shape and return
assert Lambda.shape == (len(f), len(f)), f"Expected shape for Lambda is {(len(f)}, {len(f)}), but the actual shape was {Lambda.shape}. Please check implementation"
return Lambda

# sanity check of implementation
likelihood = BernoulliLikelihood(jnp.array([0, 1, 0, 0]))
assert jnp.allclose(likelihood.log_lik(-jnp.ones((4))), -2.2530467500728912), "The implementation of the function log_lik seems to be wrong. Please check your implementation (check that the dimensionen match your expectations before summing over t
assert jnp.allclose(jnp.linalg.norm(likelihood.grad(jnp.ones((4))), 1.2944765058872572), "The implementation of the function grad seems to be wrong. Please check your implementation"
assert jnp.allclose(jnp.linalg.norm(likelihood.hessian(jnp.ones((4))), 0.3932238664829637), "The implementation of the function hessian seems to be wrong. Please check your implementation"

```

After finishing the implementation of the likelihood, we are ready to implement the *Laplace approximation* of the Gaussian process.

We will compute $\hat{\mathbf{f}}_{\text{MAP}}$ using the gradient-based optimization. To make the optimization problem easier and avoid solving the sensitive linear system $\mathbf{K}^{-1}\mathbf{f}$ in every iteration, we make use of the following reparametrization $\mathbf{f} = \mathbf{K}\mathbf{a}$ when locating the mode. That is, we let $\mathbf{f} = \mathbf{K}\mathbf{a}$ and optimize the log joint wrt. $\mathbf{a} \in \mathbb{R}^N$ and then when the optimization terminates, we compute $\mathbf{f}$ from $\mathbf{a}$.

Below you are given an incomplete implementation of the class `GaussianProcessClassification`, which uses the `BernoulliLikelihood` we implemented above.

Task 2.3: Study the class `GaussianProcessClassification` and complete the implementation of the function `construct_laplace_approximation`. The function already computes $\mathbf{f}_{\text{MAP}}$, so your task is to implement the Hessian.

Hints: Start by implementing the "straightforward" expression for S and then move on to the more robust version if you have time. The straight-forward implementation should be sufficiently accurate for reasonable hyperparameters, but might be problematic for some hyperparameter values.

```

In [ ]: probit = lambda x: norm.cdf(x)

class GaussianProcessClassification(object):

    def __init__(self, X, y, likelihood, kernel, kappa=1., lengthscale=1., jitter=1e-8):
        """
        Arguments:

```

```

        X            -- NxD input points
        y            -- Nx1 observed values
        likelihood    -- likelihood instance
        kernel        -- must be instance of the StationaryIsotropicKernel class
        jitter        -- non-negative scalar
        kappa         -- magnitude (positive scalar)
        lengthscale   -- characteristic lengthscale (positive scalar)
    """
    self.X = X
    self.y = y
    self.N = len(X)
    self.likelihood = likelihood(y)
    self.kernel = kernel
    self.jitter = jitter
    self.set_hyperparameters(kappa, lengthscale)

    # precompute kernel, its Cholesky decomposition and prepare Laplace approx
    self.K = self.kernel.contruct_kernel(self.X, self.X, jitter=self.jitter)
    self.L = jnp.linalg.cholesky(self.K)
    self.construct_laplace_approximation()

def set_hyperparameters(self, kappa, lengthscale):
    self.kernel.kappa = kappa
    self.kernel.lengthscale = lengthscale

def log_joint_a(self, a):
    """ computes and returns the log joint distribution log p(y, f), where f = K*a """
    f = self.K@a
    # compute log prior contribution
    const = -self.N/2*jnp.log(2*jnp.pi)
    logdet = jnp.sum(jnp.log(jnp.diag(self.L)))
    quad_term = 0.5*jnp.sum(a*f)
    log_prior = const - logdet - quad_term
    # compute log likelihood contribution
    log_lik = self.likelihood.log_lik(f)
    # return sum
    return log_prior + log_lik

def grad_a(self, a):
    """ computes gradient of log joint distribution, i.e. log p(y, a) = log p(y|a) + log p(a), wrt. a """
    f = self.K@a
    # compute gradient contribution from prior and likelihood
    grad_prior = -f
    grad_lik = self.likelihood.grad(f)@self.K
    # sum and return
    return grad_prior + grad_lik

def compute_f_MAP(self):
    # optimize to get f_MAP
    result = minimize(lambda a: -self.log_joint_a(a), jac=lambda a: -self.grad_a(a), x0=jnp.zeros((self.N)))

    if not result.success:
        print(result)
        raise ValueError('Optization failed')

    self.a = result.x
    f_MAP = self.K @ result.x
    return f_MAP

def construct_laplace_approximation(self):
    # f_MAP
    self.m = self.compute_f_MAP()

    Lambda = -self.likelihood.hessian(self.m)
    # Compute Hessian
    self.H = -Lambda - jnp.linalg.inv(self.K)
    self.S = jnp.linalg.inv(-self.H)

def predict_f(self, Xstar):
    """ returns the posterior distribution of f^* evaluated at each of the points in x^* conditioned on (X, y)

    Arguments:
    Xstar        -- PxD prediction points

    returns:
    mu            -- mean vector, shape (P,)
    Sigma        -- covariance matrix, shape (P, P)
    """

```

```

#####
# Your solution goes here
#####
# Covariances
K = self.K          # (N, N), from training
m, S = self.m, self.S  # (N,), (N, N)

K_star_x = self.kernel.contruct_kernel(X1=Xstar, X2=self.X, jitter=self.jitter)  # (P, N)
K_star_star = self.kernel.contruct_kernel(X1=Xstar, X2=Xstar, jitter=self.jitter)  # (P, P)

# Stable solvers (not explicit inverse)
K_inv_m = jnp.linalg.solve(K, m)
H = jnp.linalg.solve(K, K_star_x.T)

mu = K_star_x @ K_inv_m
Sigma = K_star_star - H.T @ (self.K - S) @ H
#####
# End of solution
#####

# check dimensions and return
assert (mu.shape == (len(Xstar),)), f"Expected shape for mu is {(len(Xstar))}, but the actual shape was {mu.shape}. Please check implementation"
assert Sigma.shape == (len(Xstar), len(Xstar)), f"Expected shape for Sigma is {(len(Xstar))}, {(len(Xstar))}, but the actual shape was {Sigma.shape}. Please check implementation"

return mu, Sigma

def predict_y(self, Xstar):
    """ returns the posterior distribution of y^* evaluated at each of the points in x^* conditioned on (X, y)

    Arguments:
        Xstar          -- PxD prediction points

    returns:
        p              -- vector of post. pred. probabilities, shape (P,)
    """
    #####
    # Your solution goes here
    #####
    mu_f, sigma_f = self.predict_f(Xstar)
    p = probit(mu_f / (jnp.sqrt( 8 / jnp.pi  + jnp.diag(sigma_f))))
    #####
    # End of solution
    #####

    # check dimensions and return
    assert (p.shape == (len(Xstar),)), f"Expected shape for p is {(len(Xstar))}, but the actual shape was {p.shape}. Please check implementation"
    return p

def posterior_samples(self, Xstar, num_samples):
    """
    generate samples from the posterior p(f^*|y, x^*) for each of the inputs in Xstar

    Arguments:
        Xstar          -- PxD prediction points

    returns:
        f_samples      -- numpy array of (P, num_samples) containing num_samples for each of the P inputs in Xstar
    """
    mu, Sigma = self.predict_f(Xstar)
    f_samples = generate_samples(mu.ravel(), Sigma, num_samples)

    assert (f_samples.shape == (len(Xstar), num_samples)), f"The shape of the posterior mu seems wrong. Expected {(len(Xstar))}, {num_samples}), but actual shape was {f_samples.shape}. Please check implementation"
    return f_samples

def prior_predict_f(self, Xstar):
    """
    Prior predictive for f* at Xstar (no data used).
    Returns:
        mu0      : (P,) zero vector
        Sigma0   : (P,P) K(X*, X*)
    """
    K_star_star = self.kernel.contruct_kernel(X1=Xstar, X2=Xstar, jitter=self.jitter)  # (P,P)
    mu0 = jnp.zeros((len(Xstar),), dtype=K_star_star.dtype)
    Sigma0 = K_star_star
    # sanity checks
    assert mu0.shape == (len(Xstar),), f"mu0 shape {mu0.shape} != {(len(Xstar)),}"
    assert Sigma0.shape == (len(Xstar), len(Xstar)), \
        f"Sigma0 shape {Sigma0.shape} != {(len(Xstar)), {len(Xstar))}"
    return mu0, Sigma0

def prior_predict_y(self, Xstar):

```

```

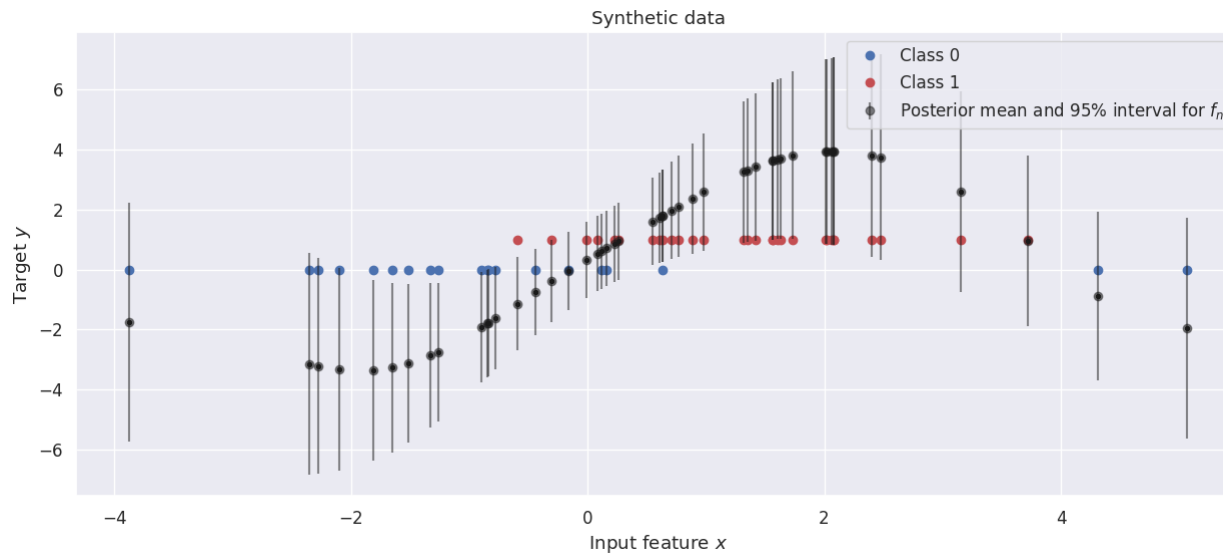
"""
Prior predictive for y* at Xstar under a probit likelihood:
p(y*=1 | X*) = E_f*[Phi(f*)] = Phi( mu0 / sqrt(1 + var0) )
For the zero-mean GP prior, this simplifies to 0.5 everywhere.
Returns:
p: (P,) prior predictive probabilities.
"""
mu0, Sigma0 = self.prior_predict_f(Xstar)
var0 = jnp.clip(jnp.diag(Sigma0), a_min=0.0)
p = probit(mu0 / jnp.sqrt(1.0 + var0)) # equals 0.5 when mu0=0
assert p.shape == (len(Xstar),), f"p shape {p.shape} != ({len(Xstar)},)"
return p

def prior_samples(self, Xstar, num_samples):
    """
    Draw samples of f* ~ N(0, K(X*,X*)) from the GP prior.
    Returns:
    f_samples: (P, num_samples)
    """
    mu0, Sigma0 = self.prior_predict_f(Xstar)
    f_samples = generate_samples(mu0.ravel(), Sigma0, num_samples)
    assert f_samples.shape == (len(Xstar), num_samples)
    return f_samples

# prep kernel, likelihood and Gaussian process
kernel = StationaryIsotropicKernel(squared_exponential)
likelihood = BernoulliLikelihood
gpc = GaussianProcessClassification(X, y, likelihood, kernel, kappa=3., lengthscale=1)

# plot data and visualize posterior
fig, ax = plt.subplots(1, 1, figsize=(15, 6))
plot_data(ax, X, y)
ax.plot(X, gpc.m, 'k.')
ax.errorbar(X, gpc.m, yerr=1.96*jnp.sqrt(jnp.diag(gpc.S)), fmt='ko', label='Posterior mean and 95% interval for $f_n$', alpha=0.5)
ax.legend();

```



If your implementation is correct, the code above visualizes the Laplace approximation superimposed on the data.

Task 2.4: Explain what you see in the plot above. That is, what does the entries of the posterior mean vector $\mathbf{m}$ represent? What about the entries in the posterior covariance matrix $\mathbf{S}$? [Discussion question]

Hints: What are the dimensions of the mean $\mathbf{m}$ and the covariance $\mathbf{S}$ of the Laplace approximation?

We approximated the intractable posterior over the latent values at the **training inputs** as

$$p(\mathbf{f} \mid \mathbf{y}, \mathbf{X}) \approx \mathcal{N}(\mathbf{m}, \mathbf{S}), \quad \mathbf{m} = \mathbf{f}_{\text{MAP}}, \quad \mathbf{S} = \left(\mathbf{K}^{-1} + \mathbf{\Lambda} \right)^{-1}, \quad \mathbf{\Lambda} = -\nabla_{\mathbf{f}}^2 \log p(\mathbf{y} \mid \mathbf{f}) \big|_{\mathbf{f}=\mathbf{m}}.$$

What the black dots (entries of $\mathbf{m}$) show:

Each m_n is the *latent function value* $f(\mathbf{x}_n)$ at training point $\mathbf{x}_n$ on the probit/logit scale, not a probability.

- $m_n > 0 \Rightarrow$ the model pushes toward class 1 (high $\Phi(m_n)$).
- $m_n < 0 \Rightarrow$ the model pushes toward class 0 (low $\Phi(m_n)$).
- $m_n \approx 0 \Rightarrow$ the model is near the decision boundary (most uncertainty in class label).

The smooth variation of the black curve across $\mathbf{x}$ is the GP prior at work.

What the vertical error bars show (diagonal of $\mathbf{S}$):

They are 95% credible intervals for each latent value:

Length $\approx$

$$1.96\sqrt{S_{nn}}$$

- **Small** S_{nn} (short bars) where labels are dense and consistent $\Rightarrow$ likelihood curvature is high and the posterior is sharp.
- **Large** S_{nn} (tall bars) where data are sparse or labels conflict $\Rightarrow$ posterior over the latent value is diffuse.

What the off-diagonals

$$S_{nm}$$

mean:

They encode posterior **correlations** between latent values at different training inputs.

- Nearby points (under the kernel) tend to have positive correlation.
- Far-apart points have correlations close to zero.
- The likelihood (through Λ) shrinks these correlations most where labels are very informative.

So, the plot is a visualization of the *latent* posterior at training inputs:

- The black dots are the approximate posterior means

$$m_n = f_{\text{MAP}}(\mathbf{x}_n)$$

.

- The bars are uncertainty from $\mathbf{S}$.
- To get class probabilities, pass the latent predictive distribution through the link (e.g., probit).
- Large

$$|m_n|$$

$\Rightarrow$ probabilities near 0 or 1.

- $m_n \approx 0 \Rightarrow$ probabilities near 0.5.

Part 3: Implementing the posterior distribution of $p(f^* | \mathbf{y}, \mathbf{x}^*)$ (step 2 of 3)

We are now ready for step 2, which is to compute the posterior distribution of the latent function evaluated at a new point $\mathbf{x}^*$, i.e to compute $p(f^* | \mathbf{y}, \mathbf{x}^*)$ where $f^* = f(\mathbf{x}^*)$. Informally, we want to be able to "fill the gaps" in the plot above.

Using the posterior approximation $q(\mathbf{f}) = \mathcal{N}(\mathbf{f} | \mathbf{m}, \mathbf{S})$, we can compute the approximate posterior for $f^* = f(\mathbf{x}^*)$ when evaluated at any $\mathbf{x}^*$ as follows:

$$p(f^* | \mathbf{y}, \mathbf{x}^*) = \int p(f^* | \mathbf{f}, \mathbf{x}^*) p(\mathbf{f} | \mathbf{y}) d\mathbf{f} \approx \int p(f^* | \mathbf{f}, \mathbf{x}^*) q(\mathbf{f}) d\mathbf{f}.$$

Since $p(\mathbf{f}, f^*)$ is a multivariate Gaussian, the conditional distribution $p(f^* | \mathbf{f}, \mathbf{x}^*)$ will also be a Gaussian, where the mean and covariance is given by the general equations on p. 84 in Murphy1 (you will derive them below).

Moreover, since $q(\mathbf{f})$ is also Gaussian, we can solve the integral above by recognizing that this integral is equivalent to a marginalization in a linear Gaussian system (see Section 3.3 in Murphy1). The results becomes

$$p(f^* | \mathbf{y}, \mathbf{x}^*) \approx \mathcal{N}(f^* | \mu_f^*, \sigma_f^2),$$

where

$$\begin{aligned}\mu_{f^*|y} &= \mathbf{k}^T \mathbf{K}^{-1} \mathbf{m}, \\ \sigma_{f^*|y}^2 &= c - \mathbf{k}^T \mathbf{K}^{-1} (\mathbf{K} - \mathbf{S}) \mathbf{K}^{-1} \mathbf{k}\end{aligned}$$

where $c = k(\mathbf{x}_*, \mathbf{x}_*)$ is the prior variance of f^* , $\mathbf{k} \in \mathbb{R}^N$ is a vector with entries given by $\mathbf{k}_n = k(\mathbf{x}_*, \mathbf{x}_n)$, and $\mathbf{K} \in \mathbb{R}^{N \times N}$ is prior covariance of $\mathbf{f}$. If you want to compute the joint distribution the predictions for P input points $\mathbf{x}^*$ at once, then $\mathbf{k}$ becomes a $P \times N$ -matrix and c becomes a $P \times P$ -matrix.

Part 3: Posterior over the latent f^* at a new input $\mathbf{x}^*$ (Step 2 of 3)

We now compute the posterior distribution of the latent function at a new point,

$$p(f^* \mid \mathbf{y}, \mathbf{x}^*) \quad \text{with} \quad f^* = f(\mathbf{x}^*).$$

Informally, this “fills the gaps” in the figure by giving a mean and uncertainty for the latent at any

$$\mathbf{x}^*$$

We will use the Laplace posterior approximation from Step 1,

$$q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, \mathbf{S}),$$

where $\mathbf{m} = \hat{\mathbf{f}}_{\text{MAP}}$ and $\mathbf{S} = (\mathbf{K}^{-1} + \mathbf{\Lambda})^{-1}$ with $\mathbf{\Lambda} = \text{diag}(\pi_n(1 - \pi_n))$ evaluated at the mode $(\pi_n = \sigma(\hat{f}_n))$.

1) Start from the GP prior joint and condition

From the GP prior,

$$\begin{bmatrix} \mathbf{f} \\ f^* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \mathbf{k} \\ \mathbf{k}^T & c \end{bmatrix}\right),$$

with

$$\mathbf{K}_{nn} = k(\mathbf{x}_n, \mathbf{x}_n), \quad \mathbf{k}_n = k(\mathbf{x}_n, \mathbf{x}^*), \quad c = k(\mathbf{x}^*, \mathbf{x}^*).$$

By Gaussian conditioning (standard formula), the conditional $p(f^* \mid \mathbf{f}, \mathbf{x}^*)$ is Gaussian:

$$f^* \mid \mathbf{f}, \mathbf{x}^* \sim \mathcal{N}(\mathbf{k}^T \mathbf{K}^{-1} \mathbf{f}, \quad c - \mathbf{k}^T \mathbf{K}^{-1} \mathbf{k}).$$

2) Integrate out $\mathbf{f}$ under the Laplace posterior

We need

$$p(f^* \mid \mathbf{y}, \mathbf{x}^*) = \int p(f^* \mid \mathbf{f}, \mathbf{x}^*) p(\mathbf{f} \mid \mathbf{y}) d\mathbf{f} \approx \int p(f^* \mid \mathbf{f}, \mathbf{x}^*) q(\mathbf{f}) d\mathbf{f}.$$

Because both are Gaussian and the conditional mean is linear in $\mathbf{f}$, the result is Gaussian:

$$f^* \mid \mathbf{y}, \mathbf{x}^* \approx \mathcal{N}(\mu_{f^*|y}, \sigma_{f^*|y}^2)$$

with

$$\begin{aligned}\mu_{f^*|y} &= \mathbf{k}^T \mathbf{K}^{-1} \mathbf{m}, & \sigma_{f^*|y}^2 &= \underbrace{(c - \mathbf{k}^T \mathbf{K}^{-1} \mathbf{k})}_{\text{conditional var given } \mathbf{f}} + \underbrace{\mathbf{k}^T \mathbf{K}^{-1} \mathbf{S} \mathbf{K}^{-1} \mathbf{k}}_{\text{uncertainty propagated from } \mathbf{f}}.\end{aligned}$$

Equivalently,

$$\sigma_{f^*|y}^2 = c - \mathbf{k}^T \mathbf{K}^{-1} (\mathbf{K} - \mathbf{S}) \mathbf{K}^{-1} \mathbf{k}.$$

\textit{Intuition:} The variance has two parts—(i) the prior uncertainty left after conditioning on a *known* $\mathbf{f}$, and (ii) extra uncertainty because $\mathbf{f}$ itself is only known approximately (with covariance $\mathbf{S}$).

3) Numerically stable forms (avoid $\mathbf{K}^{-1}$)

Using the Woodbury form of S ,

$$S = K - K\Lambda^{1/2}B^{-1}\Lambda^{1/2}K, \quad B = I + \Lambda^{1/2}K\Lambda^{1/2},$$

we get a stable expression for the variance:

$$\sigma_{f^*, y}^2 = c - (\Lambda^{1/2}k)^T B^{-1} (\Lambda^{1/2}k).$$

This avoids forming K^{-1} for the variance and uses only solves with B , which is well-conditioned (eigenvalues ≥ 1). For the mean, compute $K^{-1}m$ via a Cholesky solve with K (never by explicit inversion).

4) Multiple test points at once

For P test inputs, replace:

$$k \in \mathbb{R}^N \rightarrow K_{*} \in \mathbb{R}^{N \times P}, \quad c \in \mathbb{R} \rightarrow K_{*}^T \in \mathbb{R}^{P \times P}.$$

Then

$$\begin{aligned} \mu_{*} &= K_{*}^T K^{-1} m \in \mathbb{R}^P, \\ V_{*} &= K_{*}^T - (\Lambda^{1/2} K_{*})^T B^{-1} (\Lambda^{1/2} K_{*}) \in \mathbb{R}^{P \times P}, \end{aligned}$$

boxed{

$$\begin{aligned} \mu_{*} &= K_{*}^T K^{-1} m \in \mathbb{R}^P, \\ V_{*} &= K_{*}^T - (\Lambda^{1/2} K_{*})^T B^{-1} (\Lambda^{1/2} K_{*}) \in \mathbb{R}^{P \times P}, \\ \mu_{*} &= K_{*}^T K^{-1} m \in \mathbb{R}^P, \\ V_{*} &= K_{*}^T - (\Lambda^{1/2} K_{*})^T B^{-1} (\Lambda^{1/2} K_{*}) \in \mathbb{R}^{P \times P}, \end{aligned}$$

and individual variances are the diagonal entries of V_{*} .

Before implementing this part, we will elaborate a bit on some of the details above.

Task 3.1: Write the expression for the prior mean and the prior covariance matrix for the joint Gaussian distribution $p(f^*, f)$ in terms of the kernel quantities K , k and k .

Hints: What is the prior mean of f and f^* ? What is the prior variance of f and f^* ? What is the prior covariance between f and f^* ?

Task 3.1 Write the exp for the prior mean and prior covariance matrix for the joint Gaussian distribution $p(f^*, f)$ in terms of the kernel quantities K (covariance of the training inputs), k (covariance of training inputs), k (covariance of training and testing input)

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix} \right) \quad \begin{matrix} \Sigma_{11} = \text{Cov}(y_1, y_1) \\ \Sigma_{12} = \text{Cov}(y_1, y_2) \\ \Sigma_{21} = \text{Cov}(y_2, y_1) \\ \Sigma_{22} = \text{Cov}(y_2, y_2) \end{matrix} \quad \begin{matrix} \Sigma_{12} = \text{Cov}(y_1, y_2) \\ \Sigma_{21} = \Sigma_{12}^T \end{matrix} \quad (3.26)$$

$$p(f^*, f) = \begin{pmatrix} f^* \\ f \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} k & k^T \\ k^T & K \end{pmatrix} \right) = \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} k(x^*, x^*) & k(x^*, x) \\ k^T(x^*, x) & K(x, x) \end{pmatrix} \right)$$

Note: The next task will make you of your solution from the previous task, so you might want to verify it before continuing.

Task 3.2: Write the expression for the mean and covariance of the conditional distribution $p(f^* | f, x^*)$ using the solution to the previous task.

Hints: Use the equations (3.26) and (3.28) on p. 84 in Murphy1.

Task 3.2 Write the expression for the mean and covariance of the conditional distribution $p(f^*|f, x^*)$ using the solution from previous task 3.1

$$p(y_1|y_2) = \mathcal{N}(y_1 | \mu_{1|2}, \Sigma_{1|2}) \quad \mu_{1|2} = \mu_1 + \Sigma_{12} \Sigma_{22}^{-1} (y_2 - \mu_2) \quad (3.28)$$

$$p(f^*|f, x) = \mathcal{N}(f^* | \mu_{f^*,f}, \Sigma_{f^*,f})$$

$$= \mathcal{N}(f^* | 0 + h K^{-1} (f - 0), K - h K^{-1} h^T)$$

$$= \mathcal{N}(f^* | h K^{-1} f, K - h K^{-1} h^T)$$

The distribution $p(f^* | y, x^*)$ given above can now be derived using eq. (3.38) from Murphy1.

Now we can find the distribution $p(f^* | y, x^*)$ which is the posterior predictive distribution of the latent function value f^* at new input x^* . In GP we model a latent function $f(x)$ w/ GP and then derive a class probability, i.e. $p(y^*|x^*)$ which is what we want. $p(y^*|x^*)$ depends on values of the latent function f possible at x^* after seeing the training data y .

We use the equation:

$$p(y) \int \mathcal{N}(z | \mu_z, \Sigma_z) \mathcal{N}(y | Wz + b, \Sigma_y) dz = \mathcal{N}(y | W\mu_z + b, \Sigma_y + W\Sigma_z W^T) \quad (3.38)$$

$$p(f^*|y, x^*) = \int p(f^*|f, x^*) q(f) df$$

$$= \int \mathcal{N}(f^* | f, x^*) \mathcal{N}(f | m, S) df$$

$$= \mathcal{N}(y^* | h K^{-1} m, h - h K^{-1} h^T + h K^{-1} S h K^{-1})$$

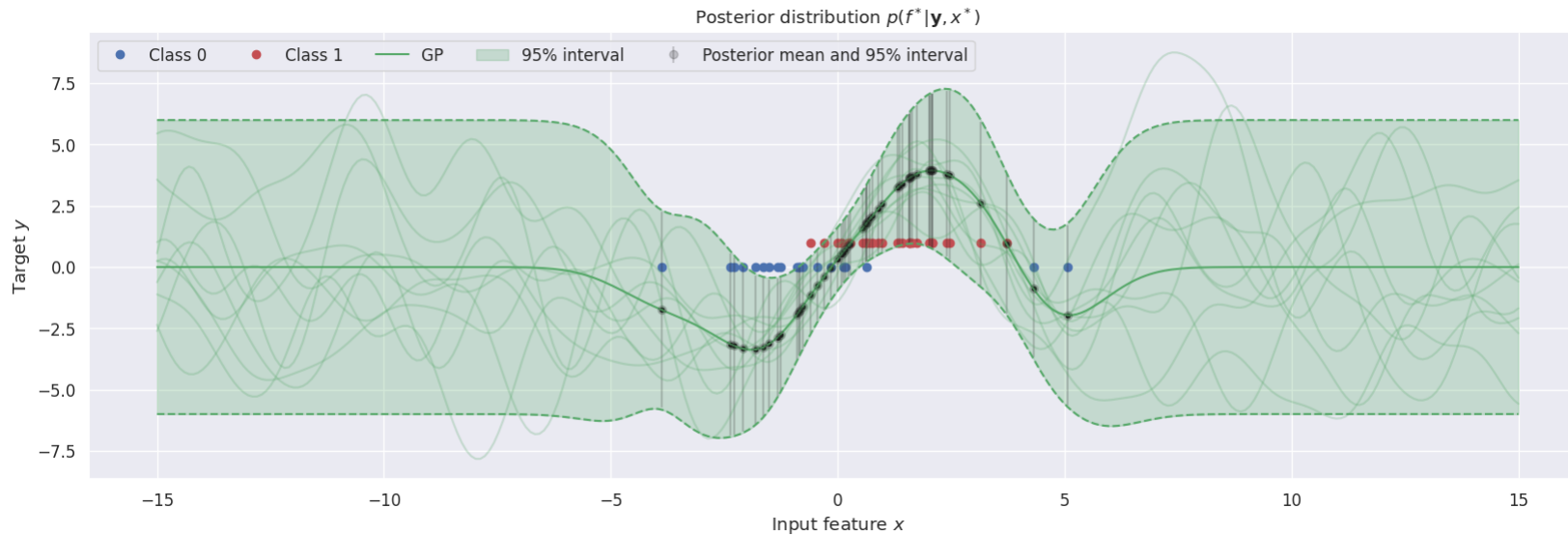
We are now ready to implement the distribution $p(f^* | y, x^*)$.

Task 3.3: Go back up the class `GaussianProcessClassification` and complete the implementation of the function `predict_f` using the equations above.

```
In [15]: # prep model
kernel = StationaryIsotropicKernel(squared_exponential)
likelihood = BernoulliLikelihood
gpc = GaussianProcessClassification(X, y, likelihood, kernel, kappa=3, lengthscale=1)

# predict
mu, Sigma = gpc.predict_f(Xstar)
```

```
# plot
fig, ax = plt.subplots(1, 1, figsize=(20, 6))
plot_data(ax, X, y)
ax.plot(X, gpc.m, 'k.')
ax.errorbar(X, gpc.m, yerr=1.96*jnp.sqrt(jnp.diag(gpc.S)), fmt='ko', label='Posterior mean and 95% interval', alpha=0.2)
plot_with_uncertainty(ax, Xstar, mu, Sigma, color='g', title='Posterior distribution $p(f^*|\mathbf{y}, \mathbf{x}^*)$', num_samples=10)
ax.legend(ncol=5);
```



Task 3.4: Play around with the kernel and hyperparameter settings and describe how they affect the resulting plot [Discussion question]

Part 4: Compute the posterior predictive distribution of the label y^* for the input point $\mathbf{x}^*$, i.e. to compute $p(y^* | \mathbf{y}, \mathbf{x}^*)$. (Step 3 of 3)

Finally, after having obtained the distribution $p(f^* | \mathbf{y})$, we can use this distribution to compute the posterior predictive distribution $p(y^* | \mathbf{y}, \mathbf{x}^*)$

$$p(y^* = 1 | \mathbf{y}, \mathbf{x}^*) \approx \int \sigma(f^*) p(f^* | \mathbf{y}) df^*.$$

We will use the **probit approximation**, which approximates the sigmoid as follows: $\sigma(f^*) \approx \Phi\left(f^* \sqrt{\frac{\pi}{8}}\right)$, where Φ is the cumulative distribution function of the standard normal distribution. The benefit is that the expectation value of the approximation can be computed analytically as follows:

$$p(y^* = 1 | \mathbf{y}, \mathbf{x}^*) \approx \int \sigma(f^*) p(f^* | \mathbf{y}, \mathbf{x}^*) df^* \approx \int \Phi\left(f^* \sqrt{\frac{\pi}{8}}\right) p(f^* | \mathbf{y}, \mathbf{x}^*) df^* = \Phi\left(\frac{\mu_f \cdot \sqrt{\frac{\pi}{8}}}{\sqrt{\frac{\sigma_f^2}{\pi} + \sigma_f^2}}\right)$$

Part 4: Posterior predictive for the label y^* (Step 3 of 3)

Having obtained the approximate marginal over the latent at a test input,

$$f^* | \mathbf{y}, \mathbf{x}^* \approx \mathcal{N}(\mu_f, \sigma_f^2),$$

we want the class-1 probability

$$p(y^* = 1 | \mathbf{y}, \mathbf{x}^*) = \mathbb{E}_{f^* | \mathbf{y}, \mathbf{x}^*} [\sigma(f^*)].$$

Probit approximation to the logistic link

Use the standard approximation of the logistic sigmoid by a scaled probit:

$$\sigma(a) \approx \Phi(\kappa a), \quad \kappa = \sqrt{\frac{\pi}{8}}.$$

Under this approximation the remaining 1-D integral is **analytic**. If $f^* \sim \mathcal{N}(m_*, v_*)$, then

$$\mathbb{E}\left[\Phi(\kappa f^*)\right] = \Phi\left(\frac{\kappa m_*}{\sqrt{1 + \kappa^2 v_*}}\right).$$

Therefore,

$$p(y^* = 1 \mid \mathbf{y}, \mathbf{x}^*) \approx \Phi\left(\frac{\kappa m_*}{\sqrt{1 + \kappa^2 v_*}}\right), \quad \kappa = \sqrt{\pi/8}$$

One-line derivation intuition

Why this identity holds (one-liner) Let $Z \sim \mathcal{N}(0, 1)$ independent of f^* . Then:

$$\mathbb{E}\left[\Phi(\kappa f^*)\right] = \Pr(Z \leq \kappa f^*) = \Pr(Z - \kappa f^* \leq 0).$$

Since $Z - \kappa f^* \sim \mathcal{N}(-\kappa m_*, 1 + \kappa^2 v_*)$, the probability equals

$$\Phi\left(\frac{\kappa m_*}{\sqrt{1 + \kappa^2 v_*}}\right).$$

Intuition

Intuition: As the uncertainty v_* grows, the denominator increases and the argument of $\Phi(\cdot)$ shrinks toward 0, pulling the predictive probability toward 0.5. When $v_* = 0$, this reduces to $\Phi(\kappa m_*) \approx \sigma(m_*)$, consistent with the link approximation.

Task 4.1: Go back up the class `GaussianProcessClassification` and complete the implementation of the function `predict_y` using the probit approximation.

```
In [16]: # seed
key = random.PRNGKey(123)

# prep model
kernel = StationaryIsotropicKernel(squared_exponential)
likelihood = BernoulliLikelihood
gpc = GaussianProcessClassification(X, y, likelihood, kernel, kappa=3., lengthscale=1.)

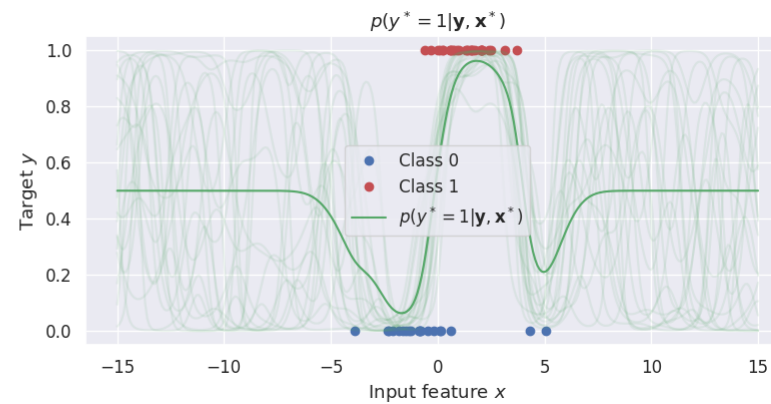
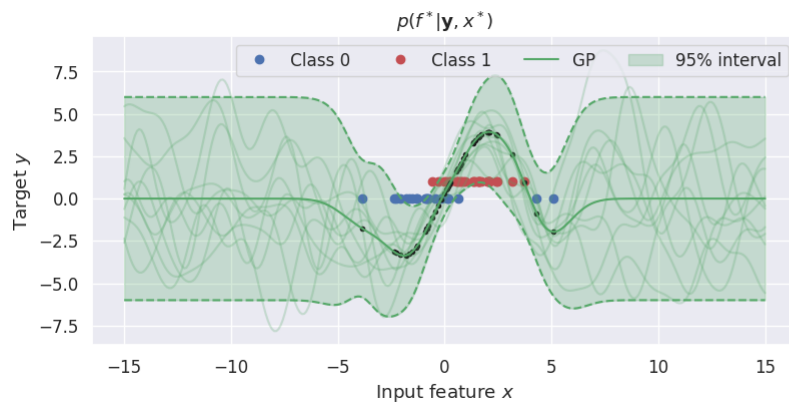
# predict
mu, Sigma = gpc.predict_f(Xstar)
p = gpc.predict_y(Xstar)

# compute prediction distribution p(f^*[y, x^*]) and p(y^*[y, x^*])
f_samples = generate_samples(key, mu[:, None], Sigma, 20).T
p_samples = sigmoid(f_samples)

# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 4))
plot_data(ax[0], X, y)
plot_data(ax[1], X, y)

ax[0].plot(X, gpc.m, 'k.')
plot_with_uncertainty(ax[0], Xstar, mu, Sigma, color='g', title='$p(f^*|\mathbf{y}, \mathbf{x}^*)$', num_samples=10)
ax[0].legend(ncol=5)

ax[1].plot(Xstar, p, 'g-', label='$p(y^*=1|\mathbf{y}, \mathbf{x}^*)$')
ax[1].plot(Xstar.ravel(), p_samples.T, 'g-', alpha=0.1)
ax[1].legend()
ax[1].set(title='$p(y^*=1|\mathbf{y}, \mathbf{x}^*)$');
```



If the code has been implemented correct, the code above produces plot of the posterior distribution $p(f^* | y, x^*)$ and the posterior predictive $p(y^* = 1 | y, x^*)$

Task 4.2: Inspect the plots above. Comment on the behavior of the posterior predictive distribution. Which regions of the input space is dominated by aleatoric uncertainty and which are dominated by epistemic uncertainty?

How to Read the Two Panels

Left panel: Latent GP, $p(f^* | y, x^*)$

- The **green band** is **epistemic uncertainty only** (uncertainty about the latent function).
- The band **balloons** where we don't have data:
 - Far left/right** (roughly $x \lesssim -5$ and $x \gtrsim 5$) and in **gaps between clusters** → **epistemic-dominated**.
 - Where points are **dense** and **labels are consistent** (e.g., around the two clusters near the center), the band is **tight** → little epistemic uncertainty.

Right panel: Class probability, $p(y^* = 1 | y, x^*)$

Here uncertainty mixes **two sources**:

1. Aleatoric-dominated:

- Near **decision boundaries** where the curve crosses ≈ 0.5 (visually, valleys between class-0 and class-1 clusters).
- The mean probability sits near **0.5**, and the latent band on the left is already fairly **tight**.
- This reflects **intrinsic overlap / label noise**: even perfect knowledge of f gives ≈ 50 -50 outcomes.

2. Epistemic-dominated:

- In the **tails** and **gaps** (same regions as above where the green band is wide).
- The fan of green sample curves **spreads widely**, and the mean drifts back toward **0.5**.
- This is due to **lack of data**; would shrink if you collected points there.

Quick Rule of Thumb

- Wide band** (left panel) ⇒ **epistemic**.
- Right panel near 0.5 with tight left-band** ⇒ **aleatoric**.
- Right panel near 0 or 1 with tight bands** ⇒ **low total uncertainty** (both sources small).

How to Reduce Uncertainty

- Reduce epistemic:** Add data where the left-panel band is wide.
- Reduce aleatoric:** Improve features/labels — more data at the same x won't remove the 0.5 ambiguity.

Short Version: Two Distributions, Not a Single Label

1. Latent function at test inputs

We first predict the GP's latent score f^* (on the probit/logit scale):

$$p(f^* \mid y, X, X^*) \approx \mathcal{N}(\mu_*, \Sigma_*).$$

- This is what you see on the **left panel** (green mean band and samples).
- It reflects **epistemic uncertainty only**.

2. Class probability at test inputs

Labels are **Bernoulli** with a link to the latent. For the probit link:

$$p(y = 1 \mid f) = \Phi(f).$$

We predict:

$$p(y^* = 1 \mid y, X, x^*) = \int \Phi(f^*) \mathcal{N}(f^* \mid \mu_*, v_*) df^* \approx \Phi\left(\frac{\mu_*}{\sqrt{1 + v_*}}\right),$$

where $v_* = \text{diag}(\Sigma_*)$.

- The **green curve** on the **right panel** is this probability.
- The **faint lines** are samples obtained by drawing f^* from the left panel and mapping through $\Phi(\cdot)$.

Summary

- `predict_f(Xstar)` $\rightarrow (\mu_*, \Sigma_*)$: posterior over the latent score.
- `predict_y(Xstar)` $\rightarrow p$: posterior class probability.
- For a **hard label**:

$$\hat{y}^* = \mathbf{1}[p \geq 0.5].$$

For comparison, we will set-up a simple neural network model as a baseline. We will use a two layer network with 20 neuron in each hidden layer. We will impose an i.i.d. Gaussian prior on all the weights, i.e. $p(w_i) \sim \mathcal{N}(0, \alpha^{-1})$ and use a MAP-estimator for the weights. We will not dive into the details of this model, but simply use it as a baseline. The class `NeuralNetworkMAP` implements estimator a MAP-estimator for the neural network model and the code can be found in the module called `exercise6.py` if you are curious.

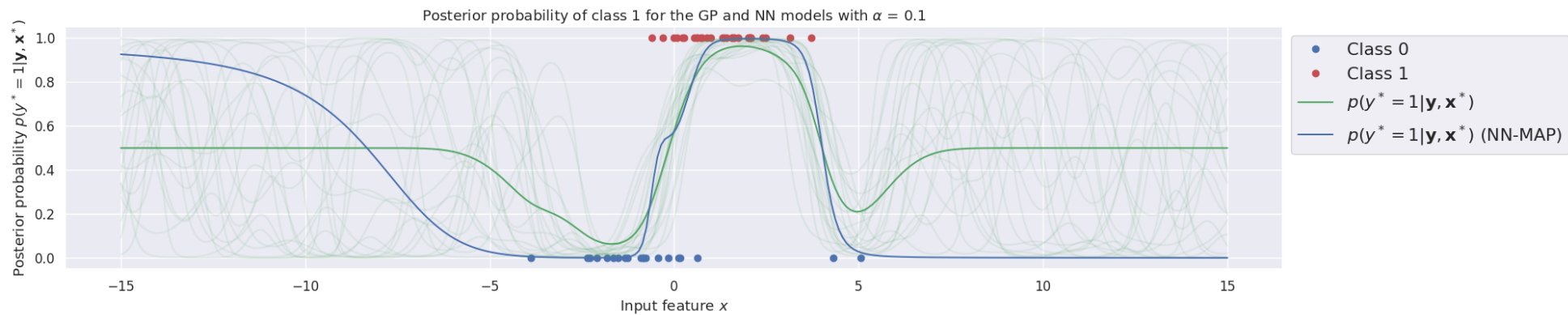
The cell below shows the predictive distribution for the network obtained using the plugin approximation compared with the Guassian process:

```
In [22]: # fit the NN model with 1 input dimension, 2 hidden layers of 20 neurons each, and 1 output
nn = NeuralNetworkMAP(X, y, [1, 20, 20, 1], alpha=.10, likelihood=BernoulliLikelihood)

# predict using neural network
y_nn = nn.predict(Xstar)
p_nn = sigmoid(y_nn)

# plot
fig, ax = plt.subplots(1, 1, figsize=(20, 4))
plot_data(ax, X, y)
ax.plot(Xstar, p, 'g-', label='$p(y^*=1|\mathbf{y}, \mathbf{x})$')
ax.plot(Xstar.ravel(), p_samples.T, 'g-', alpha=0.1)
ax.set(title='$p(y^*=1|\mathbf{y}, \mathbf{x})$ (GP)');
ax.plot(Xstar, p_nn, label='$p(y^*=1|\mathbf{y}, \mathbf{x})$ (NN-MAP)')
# change the legend of the plot outside the cell
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, labels, loc='upper left', bbox_to_anchor=(1, 1), ncol=1, fontsize='large')
ax.set_xlabel('Input feature $x$')
ax.set_ylabel('Posterior probability $p(y^*=1|\mathbf{y}, \mathbf{x})$')
ax.set_title(rf'Posterior probability of class 1 for the GP and NN models with $\alpha$ = {nn.alpha}')

Out[22]: Text(0.5, 1.0, 'Posterior probability of class 1 for the GP and NN models with $\alpha$ = 0.1')
```



Task 4.3: Compare the predictive distribution for the neural network and the Gaussian process. How do they differ (qualitatively) within the support of the data? Outside the support of the data? [Discussion question]

Hint: Go to the top of the notebook and increase the prediction range for $\mathbf{x}^*$, e.g. from $[-7.5, 7.5]$ to $[-15, 15]$

GP vs NN-MAP: Inside vs Outside Data Support (Short Notes)

Inside the data support (between the outermost labeled points)

- Both fit the class pattern reasonably well.
- GP:** shows a **distribution of functions** (faint green lines) and a **calibrated mean** $p(y^* = 1 | \mathbf{y}, \mathbf{x}^*)$ that stays near 0 or 1 only where labels are dense/consistent; it **drifts toward 0.5** near local label overlap (**aleatoric**).
- NN-MAP:** gives a **single curve** (no posterior uncertainty). It can be **sharper/overconfident** near boundaries because there's no variance term like the GP's $\mathbf{V}_{**} = \text{diag}(\Sigma_{**})$ tempering probabilities.

Outside the data support (extend $\mathbf{x}^* \in [-15, 15]$)

- GP:** reverts to its **prior**: latent mean $\rightarrow 0 \Rightarrow p(y^* = 1 | \cdot) \rightarrow 0.5$, and the **fan widens** (epistemic $\uparrow$). This reflects the kernel's inductive bias.
- NN-MAP:** extrapolates via **last-layer linear + sigmoid**; continues the trend and often **saturates** near a constant (e.g., ≈ 0.7 far left, ≈ 0.05 far right). Can be **confidently wrong** and **does not show increasing epistemic**.

Bottom line

- Within range, shapes can look similar, but **only the GP** provides **calibrated uncertainty**.
- Outside range, **GP admits ignorance** ($\rightarrow 0.5$ with wide spread), while **NN-MAP commits** to an architecture-driven extrapolation **without uncertainty**.
- To get NN uncertainty: **Bayesian NN**, **ensembles**, **MC dropout**, or **Laplace/SGHMC** over weights.

Task 4.4: What happens if you increase α from 1 to e.g. 10 or decrease it to 0.01? Can you explain the result? [Discussion question]

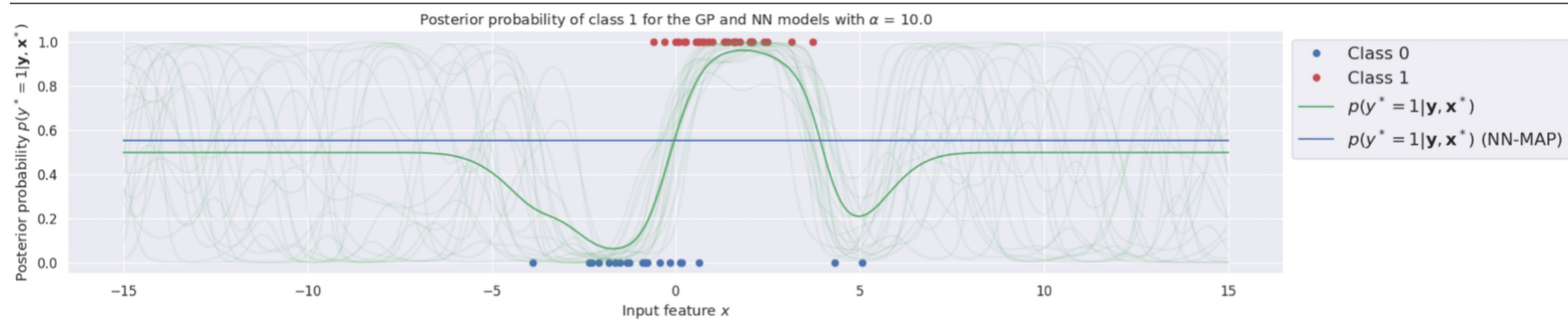
Hint: Recall the relationship between Gaussian priors and regularization when using MAP estimates



Short answer: what does α do in NN-MAP?

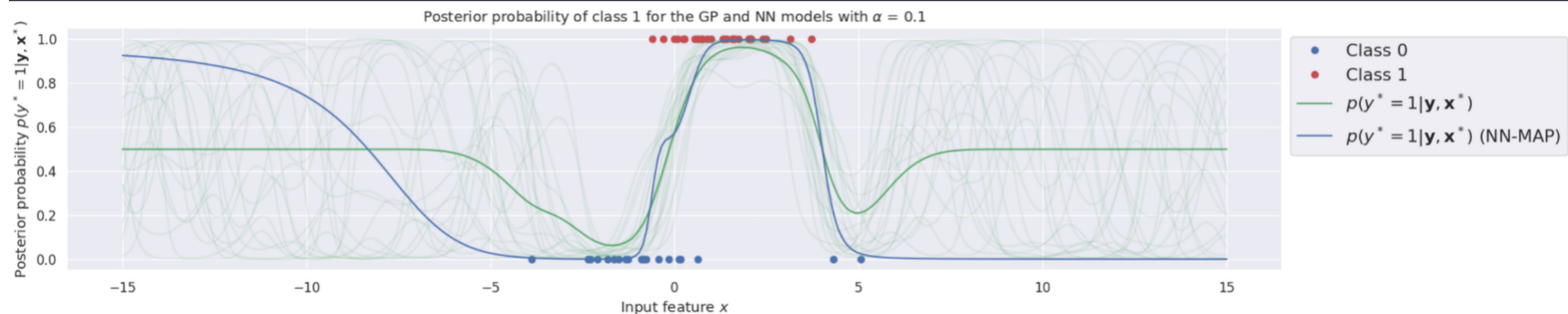
α is the L2 weight-decay strength (Gaussian prior precision).

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1} \mathbf{I}) \implies \hat{\mathbf{w}}_{\text{MAP}} = \arg \min_{\mathbf{w}} \left[-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) + \frac{\alpha}{2} \|\mathbf{w}\|_2^2 \right].$$



Increase α : $1 \rightarrow 10$ (stronger prior / more regularization)

- Weights are pushed toward 0 $\Rightarrow$ hidden units stay in their **linear regime**.
- Decision curve becomes **flatter/smooth**, peaks are **damped** $\Rightarrow$ **underfitting** risk.
- Far from data, the logit stays nearer 0 $\Rightarrow$ probabilities drift toward **0.5** (less saturation).



Decrease α : $1 \rightarrow 0.01$ (weaker prior / less regularization)

- Weights can grow $\Rightarrow$ activations **saturate** and combine **nonlinearly**.
- Curve becomes **sharper/wigglier**, fitting idiosyncrasies $\Rightarrow$ **overfitting** risk.
- Outside data, last linear layer + sigmoid **saturates** near 0 or 1 $\Rightarrow$ **overconfident** extrapolation.

Rule of thumb: higher $\alpha \rightarrow$ **smoother**, **lower-amplitude** blue curve (may miss the class-1 bump); lower $\alpha \rightarrow$ **more oscillatory**, steeper transitions, **extreme probabilities** even where there's no data.

Part 5: Gaussian process classification for 2D data set

We will now use the MNIST data from week 4 to further study the properties of Gaussian process models.

The function in the cell below loads the MNIST data and projects the data to a 2-dimensional space using PCA as we did in week 4.

```
In [23]: digits = [4, 7]
Xtrain, Xtest, ytrain, ytest = load_MNIST_subset('./mnist_subset.npz', digits=digits, subset=500)
```

```
# make sure dimensions are [N x 1]
ytrain = ytrain[:, None]
ytest = ytest[:, None]

X = jnp.concatenate((Xtrain, Xtest), axis=0)
y = jnp.concatenate((ytrain, ytest), axis=0)

# plot
def plot_data(ax, title="", xlim=(-5, 5), ylim=(-5, 5)):
    ax.plot(Xtrain[:, 0], Xtrain[:, 1], 'ko', markersize=7, label='Training data')
    ax.plot(X[y.ravel()==0, 0], X[y.ravel()==0, 1], 'b.', label='Digit %d' % digits[0])
    ax.plot(X[y.ravel()==1, 0], X[y.ravel()==1, 1], 'r.', label='Digit %d' % digits[1])
    ax.set_xlim((-5, 5))
    ax.set_ylim((-5, 5))
    ax.set(title=title, xlim=xlim, ylim=ylim, xlabel='PC1', ylabel='PC2')
    ax.legend()

fig, ax = plt.subplots(1, 1, figsize=(15, 9))
plot_data(ax, title='Subset of MNIST data')
```



Let's fit the Gaussian process

```
In [24]: # Gaussian process
kernel = StationaryIsotropicKernel(squared_exponential)
gpc = GaussianProcessClassification(Xtrain, ytrain, BernoulliLikelihood, kernel, kappa=1., lengthscale=1.)
```

... and plot the result

```
In [25]: # predict
p_train = gpc.predict_y(Xtrain)
p_test = gpc.predict_y(Xtest)

# make predictions by thresholding at p = 0.5
ytrain_hat = 1.0*(p_train > 0.5)
```

```

ytest_hat = 1.0*(p_test > 0.5)

# print results: mean and standard error of the mean
print('Training error:\t%3.2f (%3.2f)' % compute_err(ytrain_hat.ravel(), ytrain.ravel()))
print('Test error:\t%3.2f (%3.2f)' % compute_err(ytest_hat.ravel(), ytest.ravel()))

# prepare for plotting
x_grid, posterior_y_gpc_eval_mu = eval_density_grid(lambda x: gpc.predict_f(x)[0], P=50, )
x_grid, posterior_y_gpc_eval_var = eval_density_grid(lambda x: jnp.diag(gpc.predict_f(x)[1]), P=50)
x_grid, posteriorl_gpc_eval = eval_density_grid(gpc.predict_y, P=50, a=-5, b=5)

# prepare plot
fig, ax = plt.subplots(1, 3, figsize=(25, 6))

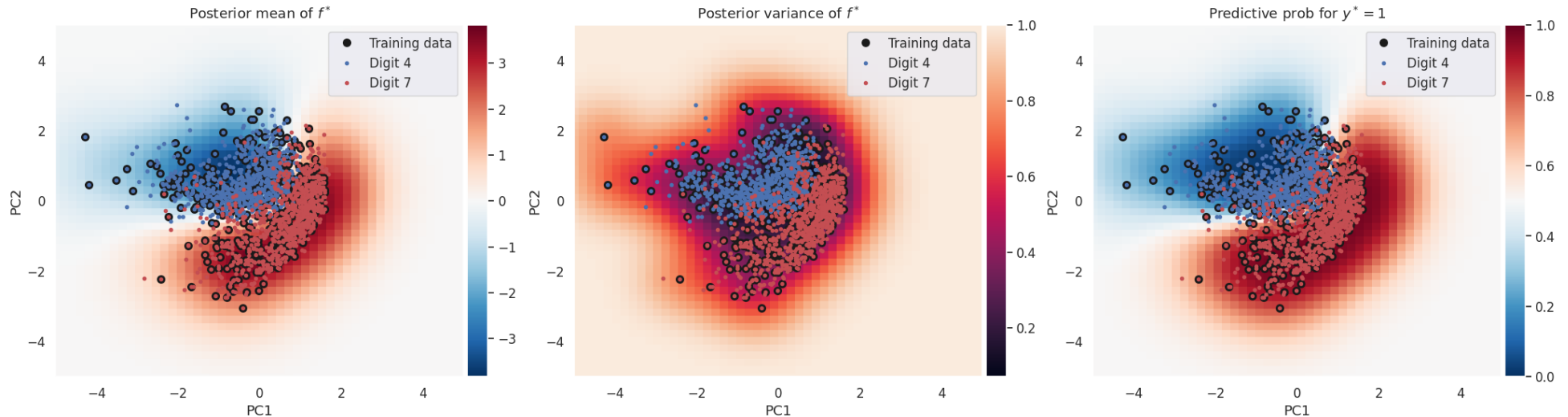
# plot posterior mean
im = ax[0].pcolormesh(x_grid, x_grid, posterior_y_gpc_eval_mu, cmap=plt.cm.RdBu_r, norm=colors.CenteredNorm(), shading='auto')
plot_data(ax[0], title="Posterior mean of $f^*$")
add_colorbar(im, fig, ax[0])

# plot posterior var
im = ax[1].pcolormesh(x_grid, x_grid, posterior_y_gpc_eval_var, shading='auto')
plot_data(ax[1], title="Posterior variance of $f^*$")
add_colorbar(im, fig, ax[1])

# plot posterior predictive
im = ax[2].pcolormesh(x_grid, x_grid, posteriorl_gpc_eval, cmap=plt.cm.RdBu_r, shading='auto', clim=(0, 1))
plot_data(ax[2], title="Predictive prob for $y^* = 1$")
add_colorbar(im, fig, ax[2])

Training error: 0.08 (0.01)
Test error: 0.08 (0.01)

```



Task 5.1: Explain what you see in the 3 panels - explain the difference between the two distributions $p(f^* | y)$ and $p(y^* | y)$ [Discussion question]

What each panel shows

Left — posterior mean of f^*

This is $\mu_{f^*}(x) = E[f^* | y, x]$ on the latent scale (probit/logit).

- **Red regions:** $\mu_{f^*}(x) > 0 \Rightarrow$ push toward digit 7.
- **Blue regions:** $\mu_{f^*}(x) < 0 \Rightarrow$ push toward digit 4.
- **Decision boundary:** pale band where $\mu_{f^*}(x) \approx 0$.

Middle — posterior variance of f^*

This is $v_{f^*}(x) = \text{Var}[f^* | y, x]$.

- **Dark (low variance):** inside dense, well-labeled regions.
- **Bright (high variance):** in gaps and far from data — pure **epistemic** uncertainty.

Right — predictive class probability

This is $p(y^* = 1 \mid y, x)$.

For a probit link, with $f^* \mid y, x \approx N(\mu_*, \nu_*)$:

$$p(y^* = 1 \mid y, x) = \mathbb{E}[\Phi(f^*)] \approx \Phi\left(\frac{\mu_*}{\sqrt{1 + \nu_*}}\right).$$

- **Red** $\approx$ class “7”, **blue** $\approx$ class “4”.
- **Grayish band near 0.5:** predicted class boundary.

The two distributions, contrasted

- $p(f^* \mid y, x)$ (left & middle):
Approximate Gaussian over the latent score.
 - μ_* tells you **which side of the boundary** you’re on.
 - ν_* says **how sure** we are about that latent value.
- $p(y^* = 1 \mid y, x)$ (right):
Bernoulli parameter from integrating the link over latent uncertainty.
 - Large ν_* pulls probabilities toward 0.5 even if μ_* is large in magnitude.
 - Small ν_* lets probabilities saturate near 0/1 when $|\mu_*|$ is large.

What you can infer from the plot

- **Inside blue/red clusters:** ν_* low $\Rightarrow$ probabilities near 0 or 1 (**confident**).
- **Near pale transition band:** $\mu_* \approx 0 \Rightarrow$ probabilities ≈ 0.5 (**aleatoric overlap**).
- **Far from data:** ν_* high $\Rightarrow$ probabilities pulled toward 0.5 (**epistemic ignorance**), even if μ_* keeps a sign.

Task 5.2: What does the values of mean **m** in the Laplace approximation $q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, \mathbf{S})$ represent and how does it relate to the figures? **[Discussion question]**

Laplace GPC: Role of **m** and **S** in Predictions

We approximate the posterior over the latent values at the **training inputs** by

$$q(\mathbf{f}) = \mathcal{N}(\mathbf{f} \mid \mathbf{m}, \mathbf{S}), \quad \mathbf{m} = \mathbf{f}_{\text{MAP}} \in \mathbb{R}^N.$$

Meaning of entries in **m**

- Each m_n is the **latent score** $f(x_n)$ at training point x_n on the **probit/logit scale** (not a probability).
- Interpretation:
 - $m_n > 0 \Rightarrow$ pushes toward **class 1**.
 - $m_n < 0 \Rightarrow$ pushes toward **class 0**.
 - $|m_n|$ = **distance from the decision boundary** in latent space.

How it appears in the figures

- **1D plot with black dots + error bars:**
 - Black dots = entries of **m**.
 - Vertical bars = $1.96\sqrt{S_{mm}}$ (95% credible intervals from the diagonal of **S**).
- **MNIST heatmaps – Posterior mean of f^* (left panel):**
 - For a test location x^* :

$$\mu_*(x^*) = K_* \chi K^{-1} \mathbf{m}.$$

- Red = $\mu_*(x^*) > 0$, Blue = $\mu_*(x^*) < 0$.

Predictive class probability (right panel)

- Uses $\mu_*(x^*)$ and predictive variance $v_*(x^*)$ from S:
 - For probit link:

$$p(y^* = 1 \mid y, x^*) \approx \Phi\left(\frac{\mu_*(x^*)}{\sqrt{1 + v_*(x^*)}}\right), \quad v_*(x^*) = \text{Var}[f^* \mid y, x^*].$$

- Large $v_*(x^*) \Rightarrow$ probability pulled toward 0.5.
- Small $v_*(x^*)$ + large $|\mu_*(x^*)| \Rightarrow$ probability near 0 or 1.

Summary:

- m** = MAP estimate of latent function at training inputs.
- Seeds the **entire predictive mean field** $\mu_*(x^*)$.
- After applying the **link function**, it produces the **probability map** in the rightmost figure.

Task 5.3: Comment on the uncertainties within the support of the data and outside the data. Compare with the equivalent plots for the NN below [Discussion question]

GP vs NN-MAP: Inside vs Outside Data Support

Within the data support

GP (Laplace GPC)

- Middle panel** ($\text{Var}[f^* \mid y, x^*]$) is low inside the two dense clouds $\Rightarrow$ little **epistemic** uncertainty there.
- Gray/whitish ribbon:** where the left panel's $\mu_*(x) \approx 0$ and the right panel's $p(y^* = 1 \mid \cdot) \approx 0.5 \Rightarrow$ **aleatoric** uncertainty (class overlap).
- Even with small variance, probabilities ≈ 0.5 when $\mu_*(x) \approx 0$.

NN-MAP

- Produces a **single curve/surface** (no posterior variance) $\Rightarrow$ cannot express **epistemic** uncertainty.
- Typically **overconfident**: probabilities saturate near 0/1 across most of the support, including near the boundary (unless very strong regularization is used).
- Cannot separate aleatoric from epistemic uncertainty.

Outside the data support (far from any training points)

GP

- Variance **inflates** (bright in the middle panel), $\mu_*(x) \rightarrow 0 \Rightarrow p(y^* = 1 \mid \cdot) \rightarrow 0.5$ in the right panel.
- This is **classic epistemic uncertainty**: the model admits ignorance away from data.

NN-MAP

- Extrapolates its learned trend, often **saturating** to 0 or 1 without increasing uncertainty.
- Can be **confidently wrong** in the tails — no mechanism to revert to 0.5.

TL;DR

- GP:**
 - Inside = **aleatoric** near the boundary, low total uncertainty in cluster interiors.
 - Outside = **epistemic** dominates (variance $\uparrow$, probability $\rightarrow 0.5$).
- NN-MAP:**
 - Inside = confident almost everywhere.
 - Outside = stays confident (often saturating), offers no uncertainty unless made Bayesian (ensembles, MC-dropout, Laplace over weights).

```
In [26]: nn = NeuralNetworkMAP(Xtrain, ytrain, [2, 20, 20, 1], alpha=1., likelihood=BernoulliLikelihood)

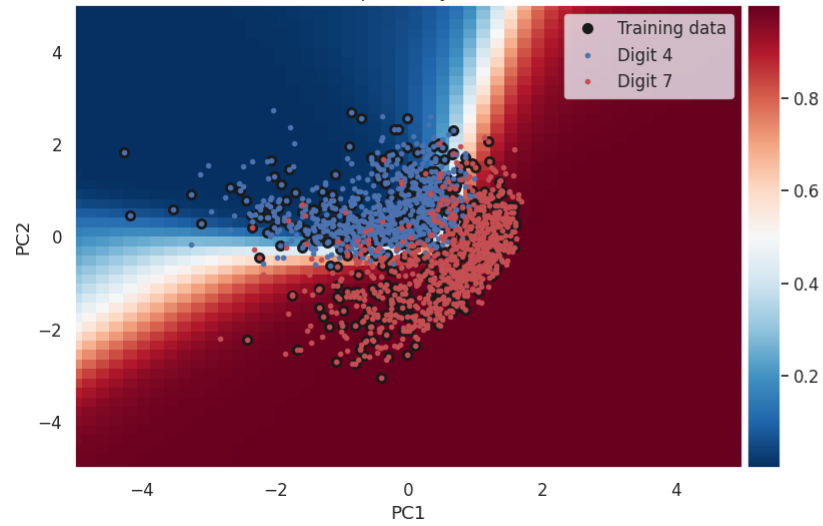
x_grid, posterior1_nn_eval = eval_density_grid(lambda x: sigmoid(nn.predict(x)), P=50, a=-5, b=5)

fig, ax = plt.subplots(1, 2, figsize=(20, 6))
im = ax[0].pcolormesh(x_grid, x_grid, posterior1_nn_eval, cmap=plt.cm.RdBu_r, shading='auto')
plot_data(ax[0], title='Predictive prob for y = 1 for NN')
add_colorbar(im, fig, ax[0])
```

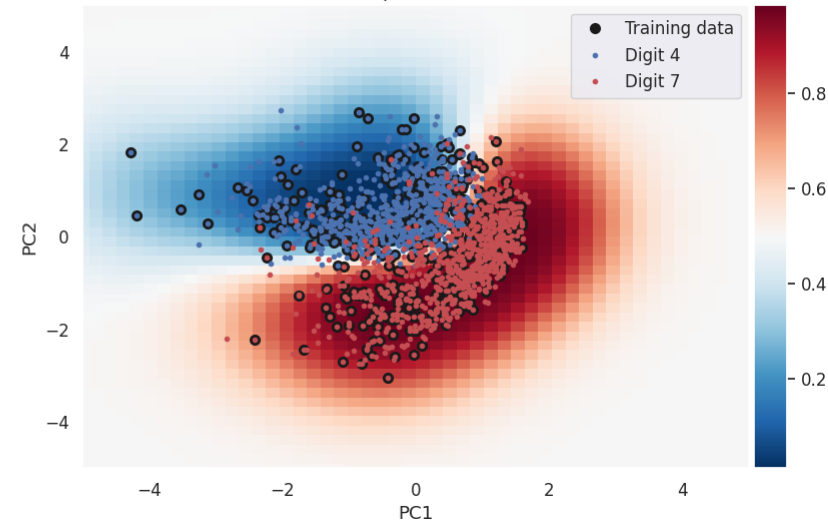
```
plot_data(ax[1], title='Posterior predictive for GP')
im = ax[1].pcolormesh(x_grid, x_grid, posterior1_gpc_eval, cmap=plt.cm.RdBu_r, shading='auto')
add_colorbar(im, fig, ax[1])
```

```
E0809 16:08:17.816056 7600 hlo_lexer.cc:443] Failed to parse int literal: 1497414723839453098837
E0809 16:08:17.816120 7600 hlo_lexer.cc:443] Failed to parse int literal: 1497414723839453098837
E0809 16:08:18.230552 7600 hlo_lexer.cc:443] Failed to parse int literal: 99966264837506927423
E0809 16:08:18.230726 7600 hlo_lexer.cc:443] Failed to parse int literal: 99966264837506927423
```

Predictive prob for y = 1 for NN



Posterior predictive for GP



In []: